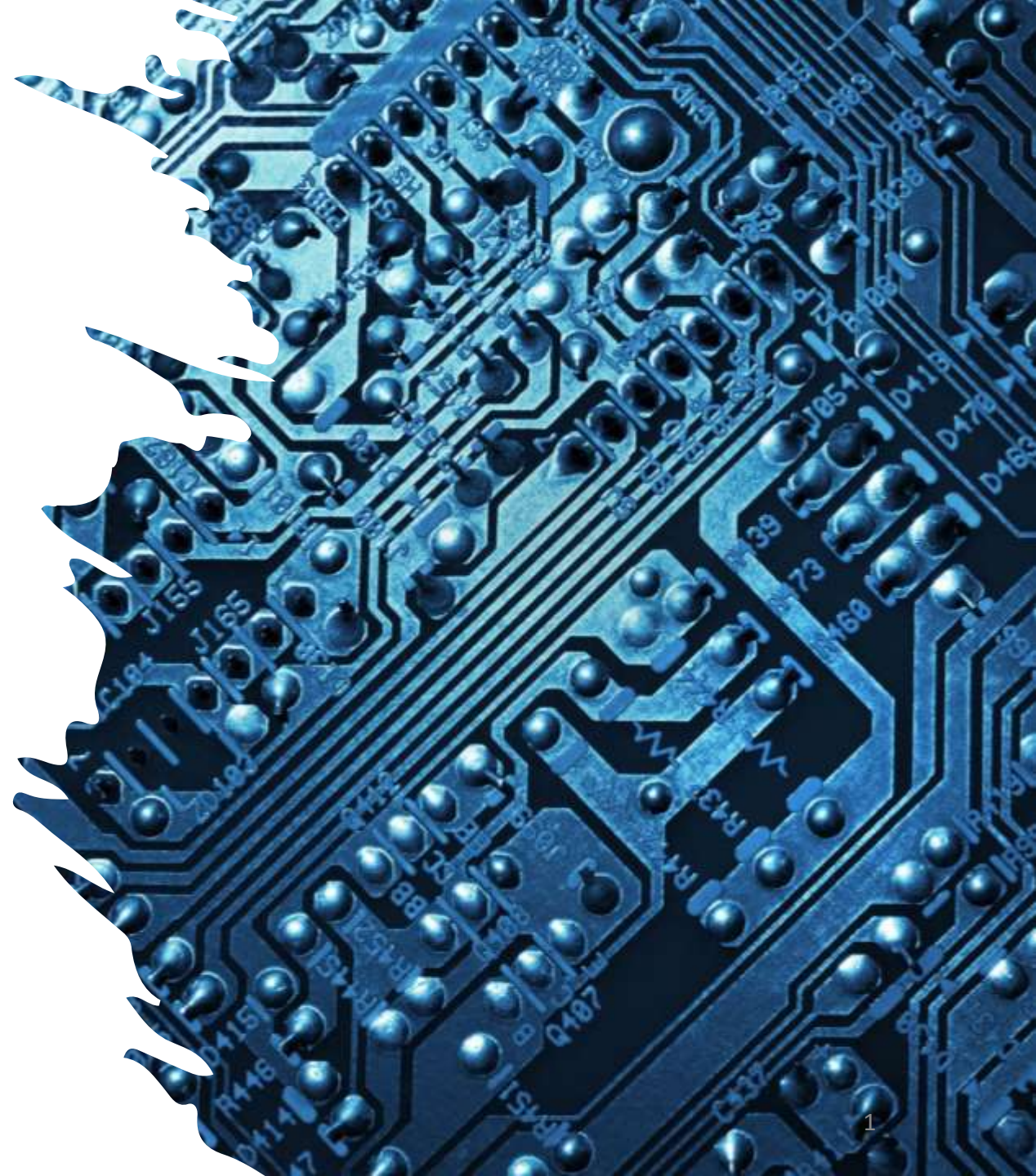
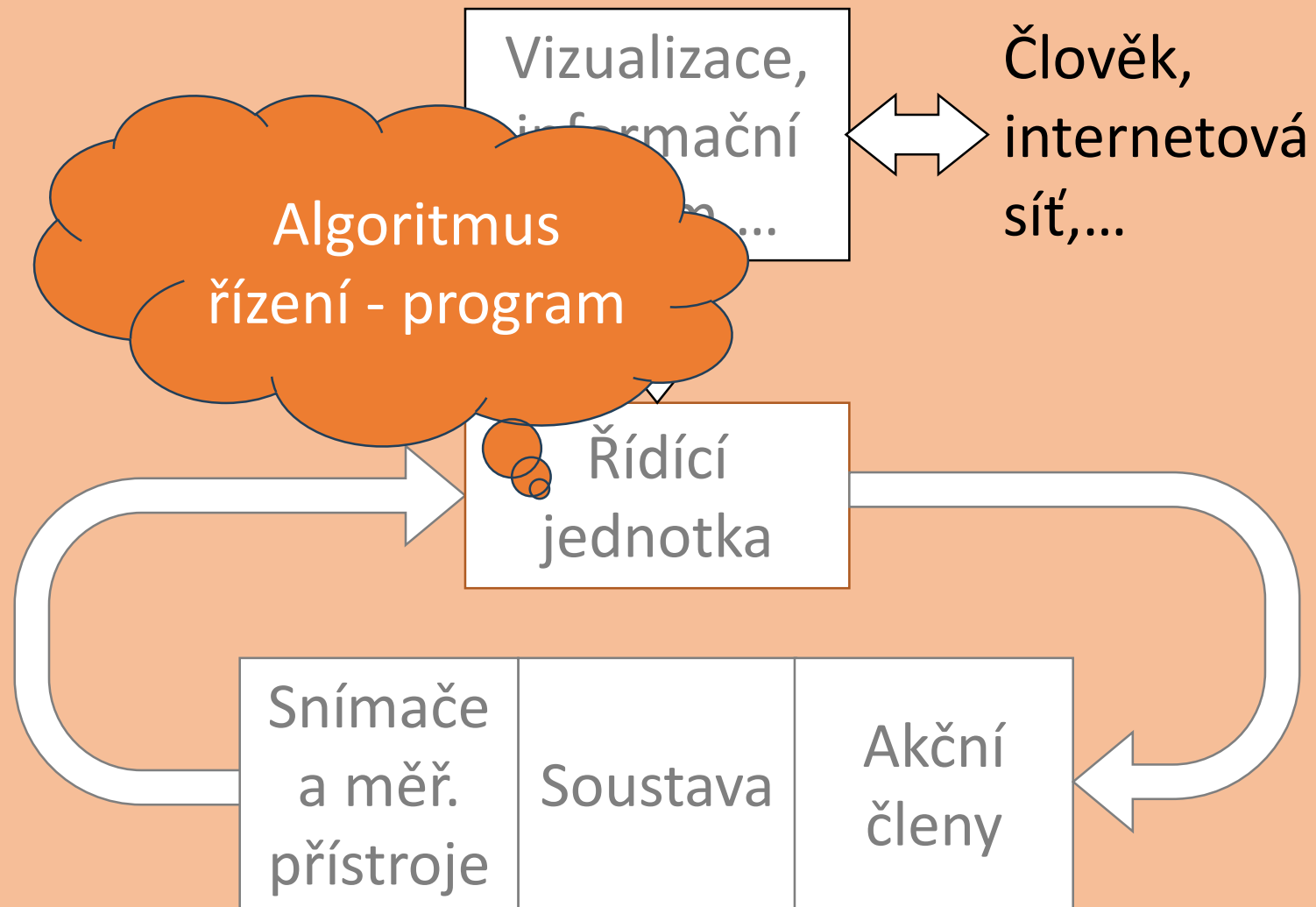


MTE a MPS

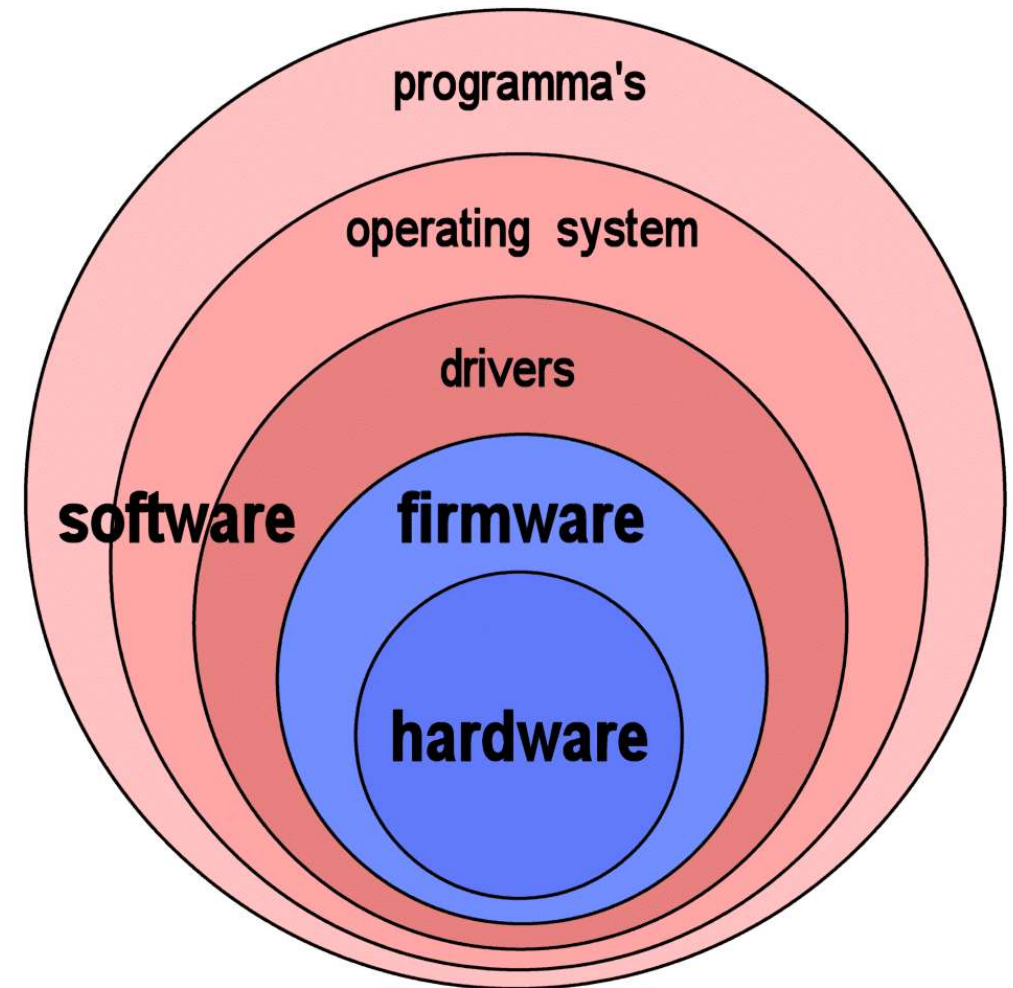
Mikroprocesorová technika
a mikropočítačové systémy





SW řídicích jednotek

- **Firmware** – sw pro řízení hw, je s ním úzce spjat, někdy podobný driverům (HDD, klávesnice...), někdy rozsáhlejší (fotoaparát...). Nejčastěji MCU.
- **OS** – sw zajišťující komunikaci člověka s hw.
- **RTOS** – Real Time Operating System – operace vykoná v pevně stanoveném časovém úseku podle priorit. Např. PLC, iPC.





Real Time Operating System

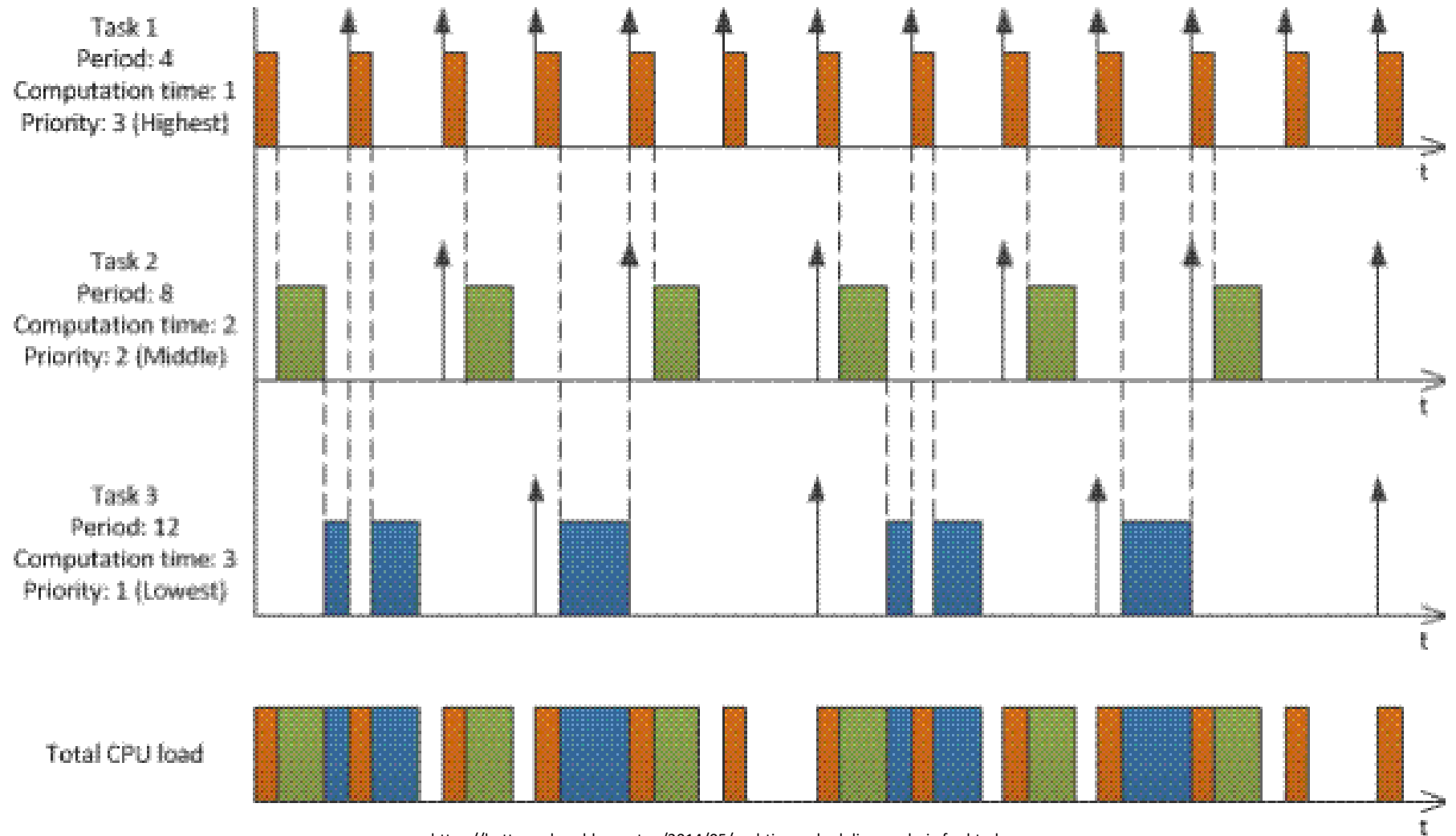
- Jak zpracovávají úlohy běžné uživatelské OS?
- Jak zpracuje jeden procesor více úloh najednou?
- Jak zpracuje jeden procesor více úloh najednou vyžadujících stejné prostředky?
- **Jak VY uděláte víc úloh najednou?**
- Dokáže procesor reagovat v reálném čase?
- **Dokážete VY reagovat v reálném čase?**
- Jak pracují RTOS?



RTOS

- Závislosti úloh
 - prostorová záv. – přidělení priorit a spec. algoritmy pro řešení
 - časová záv. (precedence constraints) – přidělení priorit a přerušování úloh
- Úlohy na sebe navazují – přidělení priorit
- Ne vždy je úloha vyžadující obsluhu právě potřeba – algoritmy přidělování priorit
- Nepravidelné úlohy – přidělení priorit, prodleva (idle time) > 0

RTOS – ukázka precedenčního grafu



Cyklus řešení uživatelského programu

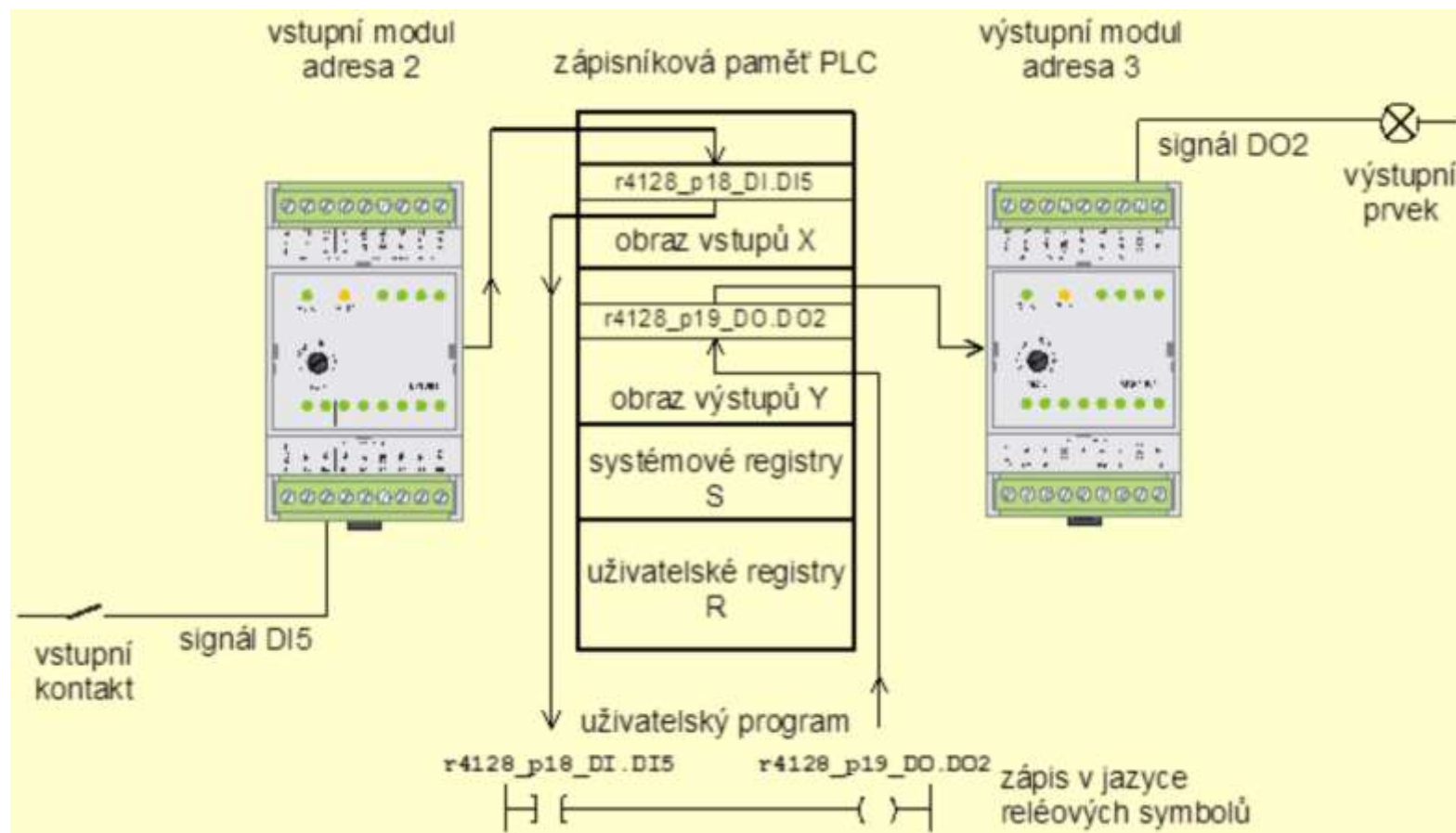
- **čtení X** - přepis hodnot ze vstupních modulů PLC do oblasti X v zápisníkové paměti
- **zápis Y** - přepis hodnot vypočtených programem z oblasti Y do výstupních modulů PLC
- **režie** - příprava centrální jednotky PLC k řešení dalšího cyklu programu



Převzato z nápovědy Mosaic, Teco

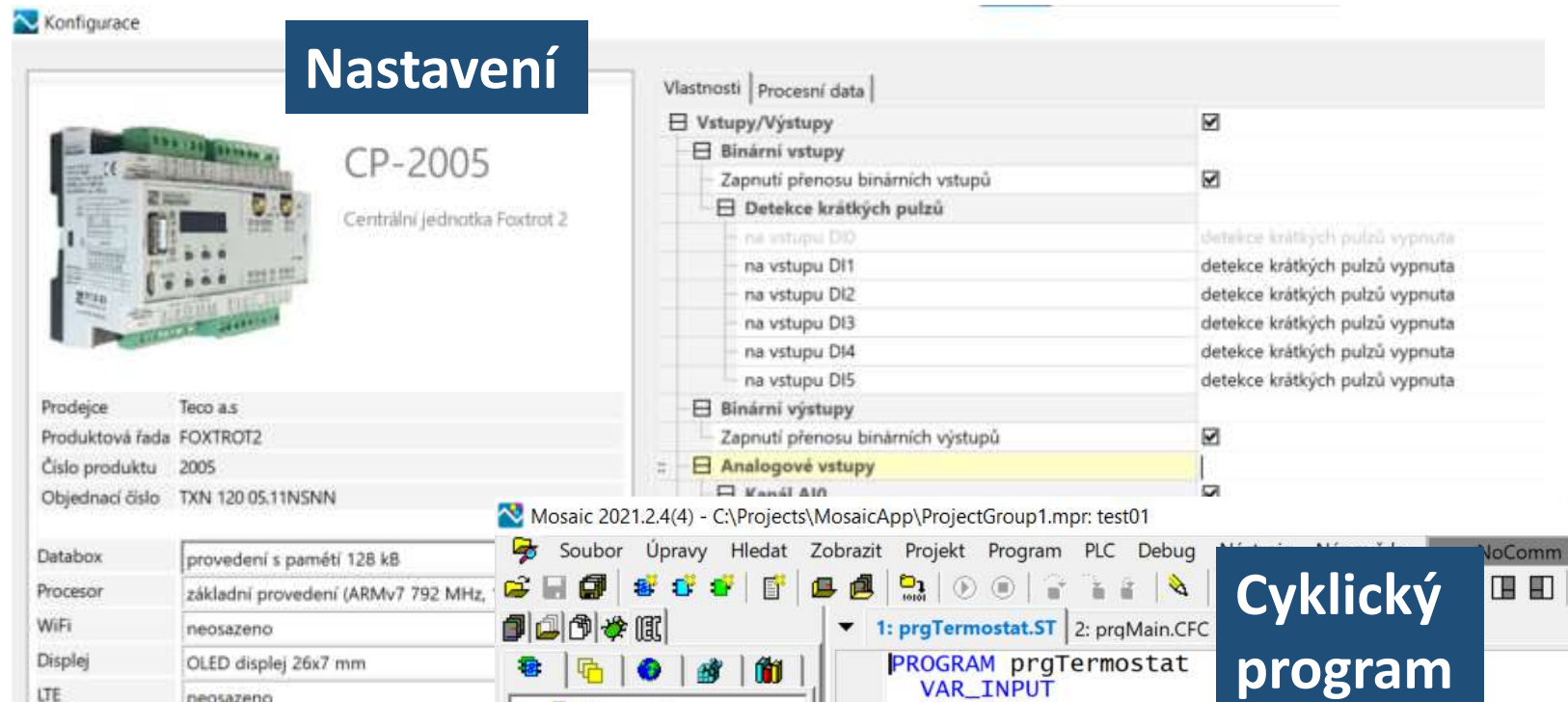
Jednorázová aktualizace stavů vstupních proměnných během celého cyklu programu odstraňuje možnosti vzniku hazardních stavů při řešení algoritmu řízení (během výpočtu nemůže dojít ke změně vstupních proměnných).

Schéma zpracování signálů programovatelným automatem



Základní části programu PLC

Nastavení



Konfigurace

CP-2005
Centrální jednotka Foxtrot 2

Prodejce: Teco a.s.
Produktová řada: FOXTROT2
Číslo produktu: 2005
Objednávací číslo: TXN 120 05.11NSNN

Databox: provedení s pamětí 128 kB
Procesor: základní provedení (ARMv7 792 MHz)
WiFi: neosazeno
Displej: OLED displej 26x7 mm
LTE: neosazeno

Vlastnosti	Procesní data
Vstupy/Výstupy	☒
Binární vstupy	☒
Zapnutí přenosu binárních vstupů	☒
Detekce krátkých pulzů	
na vstupu DI0	detekce krátkých pulzů vypnuta
na vstupu DI1	detekce krátkých pulzů vypnuta
na vstupu DI2	detekce krátkých pulzů vypnuta
na vstupu DI3	detekce krátkých pulzů vypnuta
na vstupu DI4	detekce krátkých pulzů vypnuta
na vstupu DI5	detekce krátkých pulzů vypnuta
Binární výstupy	☒
Zapnutí přenosu binárních výstupů	☒
Analogové vstupy	☒
Kanal A10	☒

Mosaic 2021.2.4(4) - C:\Projects\MosaicApp\ProjectGroup1.mpr: test01

Soubor Úpravy Hledat Zobrazit Projekt Program PLC Debug

1: prgTermostat.ST 2: prgMain.CFC

Programy

- prgMain
- prgTermostat
- Funkční bloky
- Funkce

Cyklický program

Přejmenování pinů HW konfigurace



Konfigurace

CP-2005
Centrální jednotka Foxtrot 2

Prodejce: Teco a.s.
Produktová řada: FOXTROT2
Číslo produktu: 2005

Vlastnosti	Procesní data
ITCL_DIAG : DT_ITCL_DIAG	
TCL2_DIAG : DT_TCL2_DIAG	
CIB_DIAG : DT_CIB_DIAG	
IN : DT_IR2551_IN	
DIS : TBIN_6DI	
DI0 : BOOL	
DI1 : BOOL	
DI2 : BOOL	
DI3 : BOOL	
DI4 : BOOL	
DI5 : BOOL	

```
PROGRAM prgTermostat
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
Tm : REAL; (* naměřená teplota *)
Tw : REAL := 25.0; (* požadovaná teplota *)
Topeni : BOOL; (* spínaný výstup *)

(* parametry regulátoru *)
deltaT : REAL := 1.0; (* rozdíl mezi požadovanou a naměřenou teplotou *)
Tdead : REAL := 0.5; (* mrtvá zóna regulace *)
END_VAR
VAR_TEMP
END_VAR

IF Tm < Tw - deltaT - Tdead THEN
    Topeni := TRUE; (* zapnutí topení *)
ELSIF Tm > Tw + deltaT + Tdead THEN
    Topeni := FALSE; (* vypnutí topení *)
END_IF;

END_PROGRAM
```

Základní části programu MCU

Blink§

```
1 #define LED 13
2 //int LED = 13;
3 //Jednořádkový komentář se nepřekládá
4 void setup() {
5     pinMode(LED, OUTPUT);
6 }
7 /* Ani více řádkový
8 komentář se nepřekládá*/
9 void loop() {
10     digitalWrite(LED, HIGH);
11     delay(1000);
12     digitalWrite(LED, LOW);
13     delay(1000);
14 }
```

Přejmenování pinů

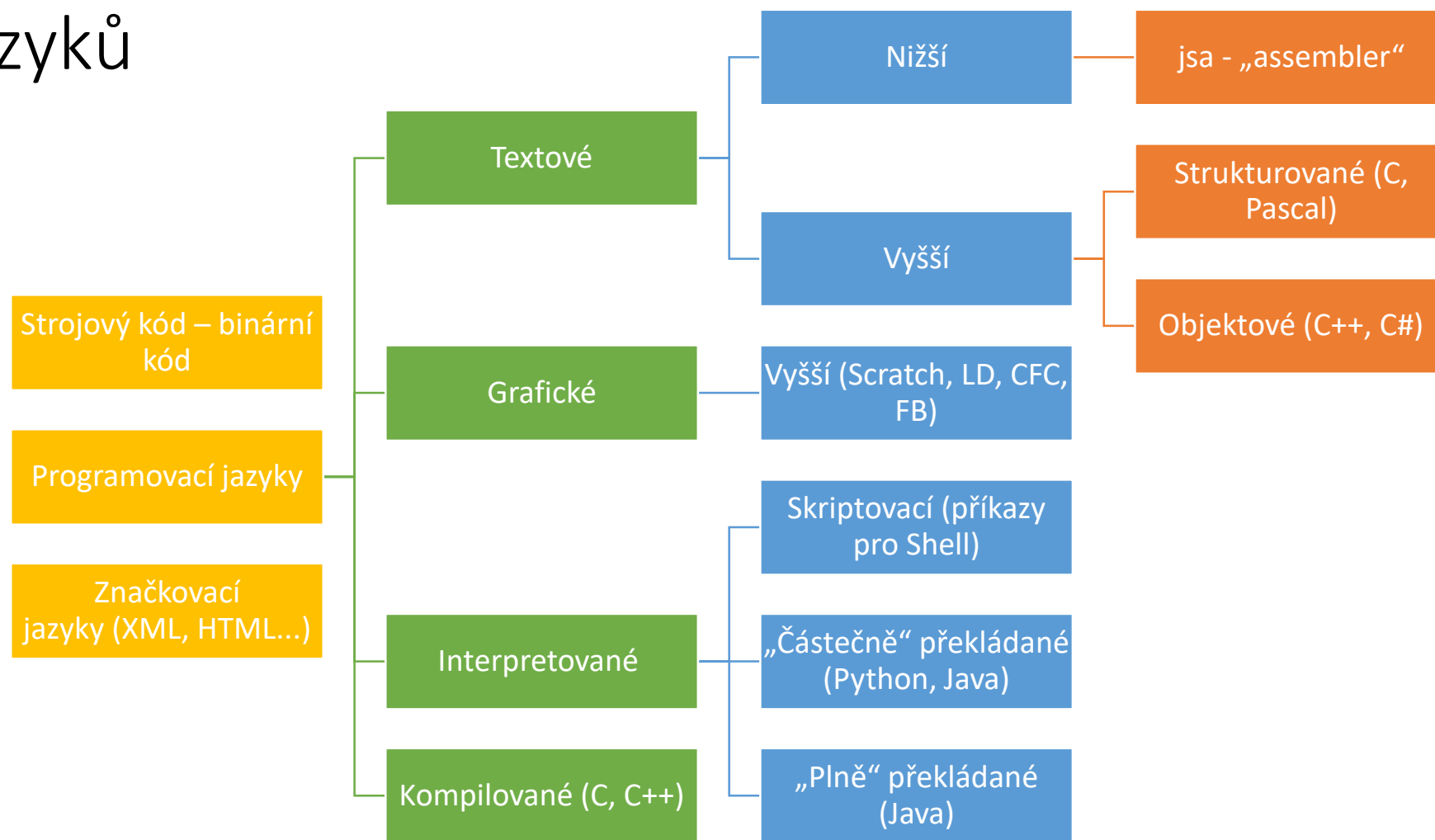
Nastavení
HW konfigurace

Cyklický
program

Algoritmus řízení

- Algoritmus – přesný návod či postup, kterým lze vyřešit daný typ úlohy
- Program = Σ instrukcí
- Instrukce = povel, příkaz
- Programovací jazyk – soubor pravidel pro zápis algoritmu

Rozdělení programovacích jazyků



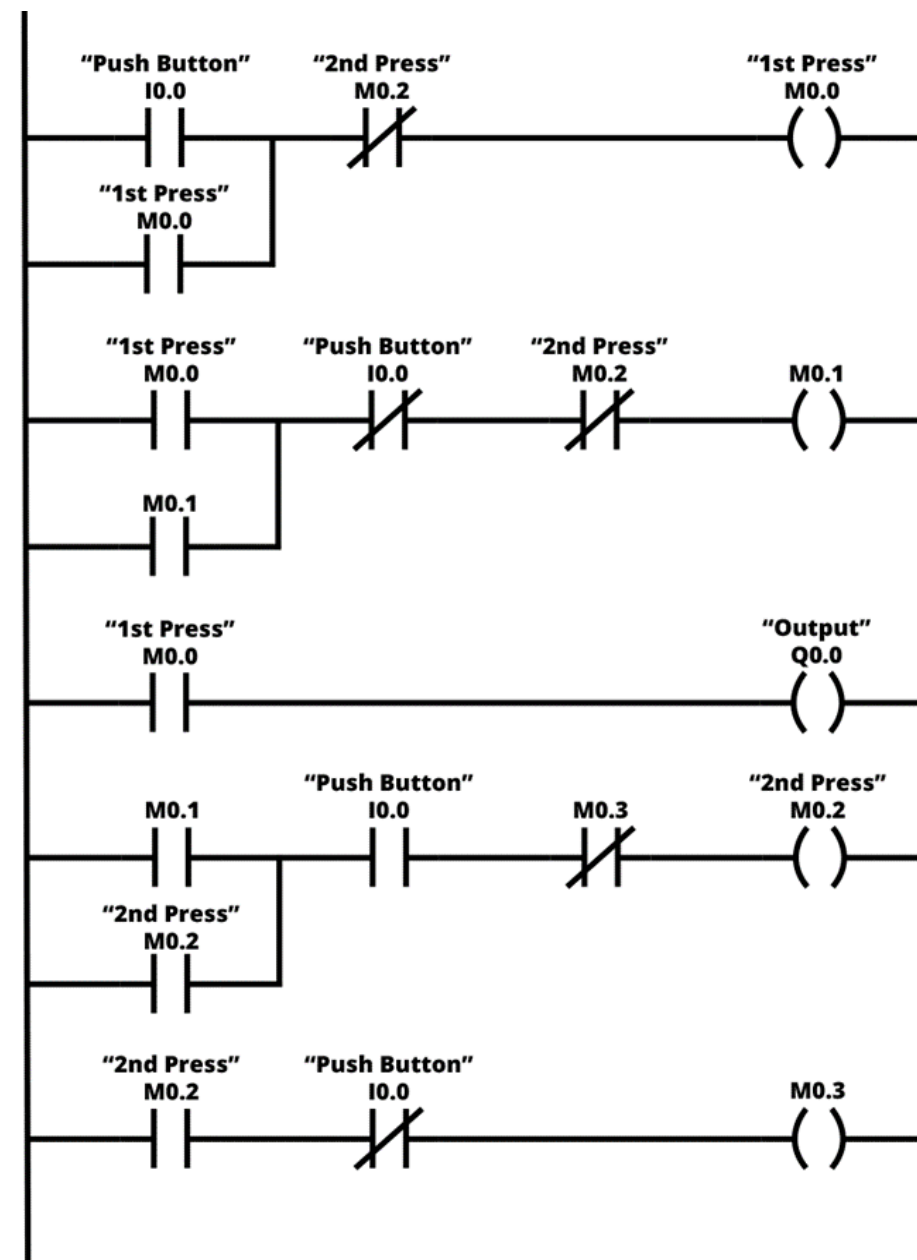
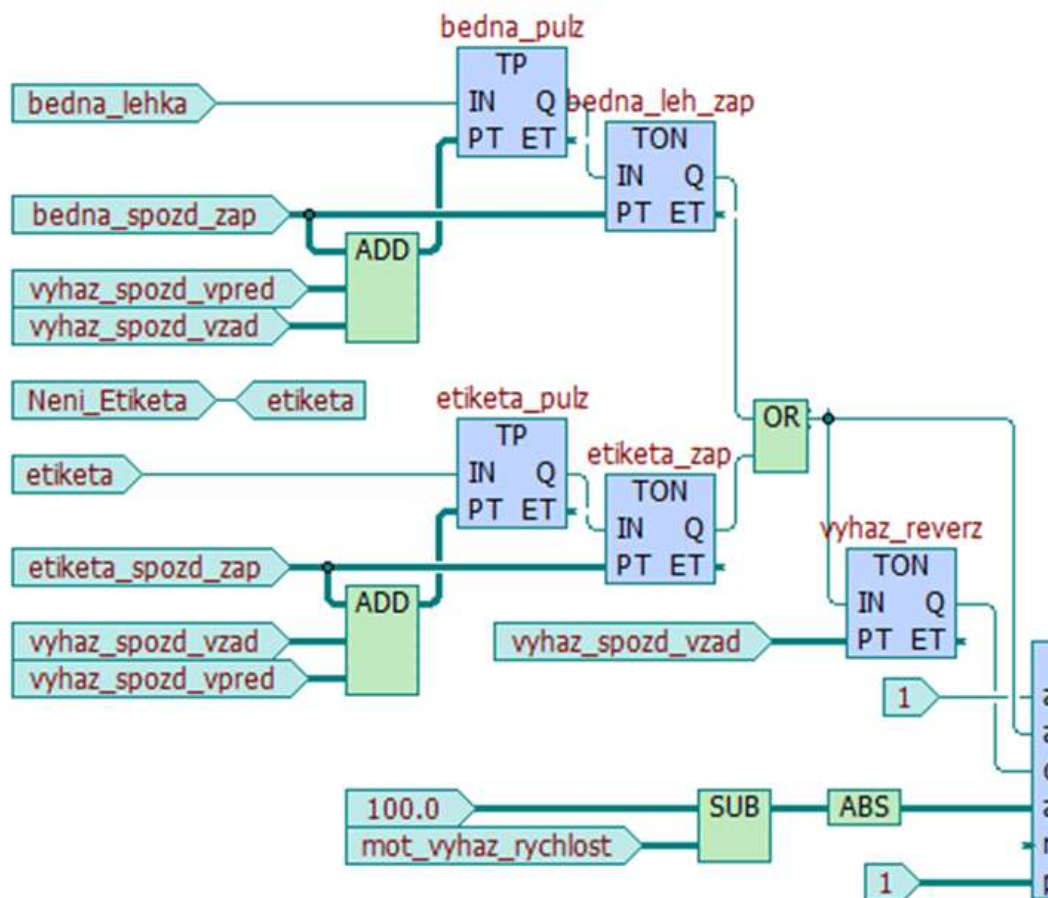
Pozn.: Pokud se pouze nastavují hodnoty, adresy,... ale nepíše se samotný program, jedná se o parametrování.

Programovací jazyky

Assembler	Funkce	Strojový kód
clrf f	Clear f	00 0001 1fff ffff
nop	No Operation	00 0000 0xx0 0000
bcf f, b	Bit Clear f	01 00bb bfff ffff

	org	0	
	bsf	STATUS, RPO	#include <stdio.h>
	bcf	TRISD, 0	
	bcf	STATUS, RPO	int main(void)
Start:	bsf	PORTD, 0	{
	bcf	PORTD, 0	printf(„No nazdar!\n“);
	goto	Start	return 0;
	end		}

Programovací jazyky



Informace X Data

1101 0011 1001 1010 0101 0011

Paměti

Kde najdeme uložená data?



Proměnné a datové typy

DATOVÝ_TYP NÁZEV_PROMĚNNÉ

- bool
- int
- float, double, real
- char, string
- ...





Proměnné

- **Lokální proměnná** – definuje se uvnitř funkce, po opuštění této funkce zaniká (místo v paměti se uvolní pro další použití)
- **Globální proměnná** – definuje se mimo všechny funkce, využívá se pro přenos dat mezi funkcemi, apod., existuje od svého definování až do ukončení celého programu
- **Statická proměnná** – definuje se uvnitř funkce klíčovým slovem **static**, ale existuje až do ukončení programu (řádek, kde je definována se provede pouze jednou, dále se ignoruje)
- (<https://www.arduino.cc/reference/en/#page-title> -> Variables)

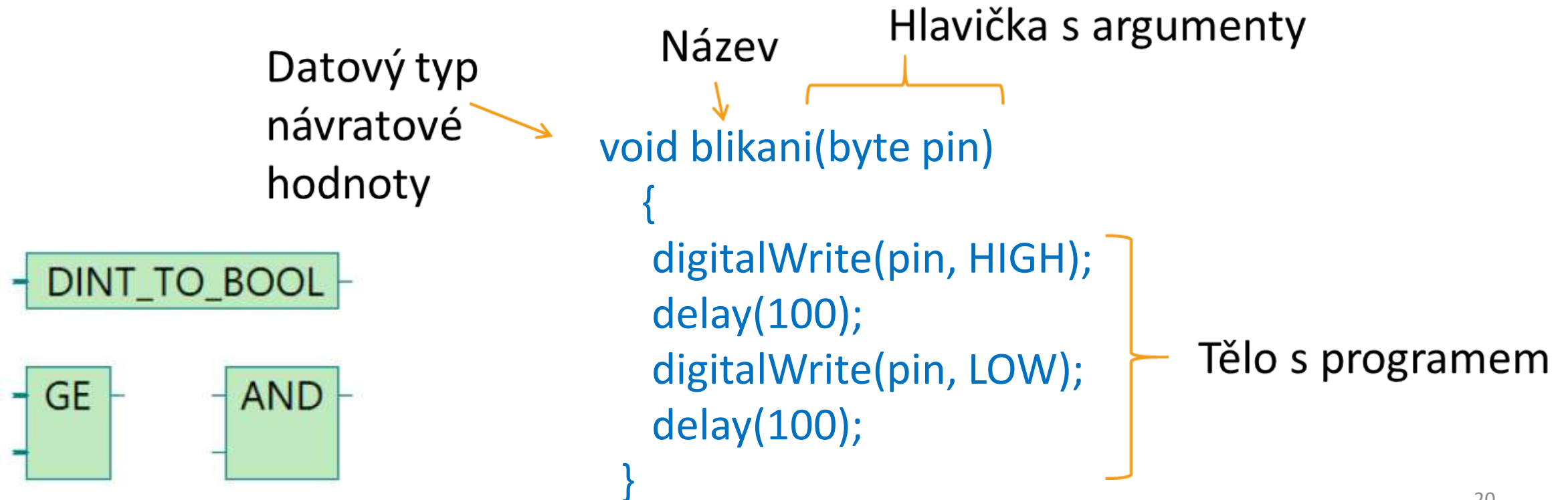


Lokální,
globální,
nebo
statickou?

```
#define LED 10
unsigned int citac = 0; //Globální proměnná
void setup() {
    pinMode(LED, OUTPUT);
}
void loop() {
    static unsigned int citac = 0; //Statická proměnná
    unsigned int citac = 0; //Lokální proměnná
    static bool stav_led = 0;
    citac++;
    if(citac >= 30000) {
        stav_led = !stav_led;
        digitalWrite(LED, stav_led);
        citac = 0;
    }
}
```

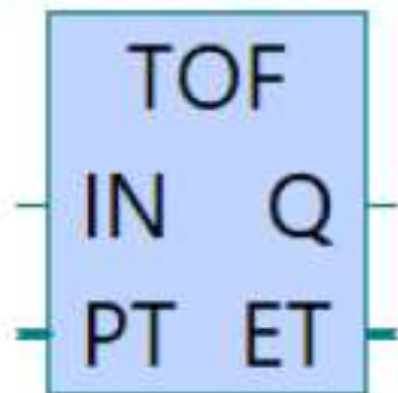
Podprogramy, funkce

- **Funkce** – slouží pro delší úsek kódu, který se často opakuje nebo ke zpřehlednění programu. Jsou buď v knihovnách, nebo si je můžeme sami nadefinovat (<https://www.sallyx.org/sally/c/c13.php>)



Objekty, funkční bloky

Svetlo_WC



```
2 public class Pen {  
3  
4     String type = "gel";  
5     String color = "blue";  
6     int point = 10;  
7  
8     static boolean clicked = false;  
9  
10    public static void click() {  
11        clicked = true;  
12    }  
13  
14    public static void unclick() {  
15        clicked = false;  
16    }  
17  
18 }
```

[maxresdefault.jpg \(1280×720\) \(yting.com\)](https://maxresdefault.jpg)



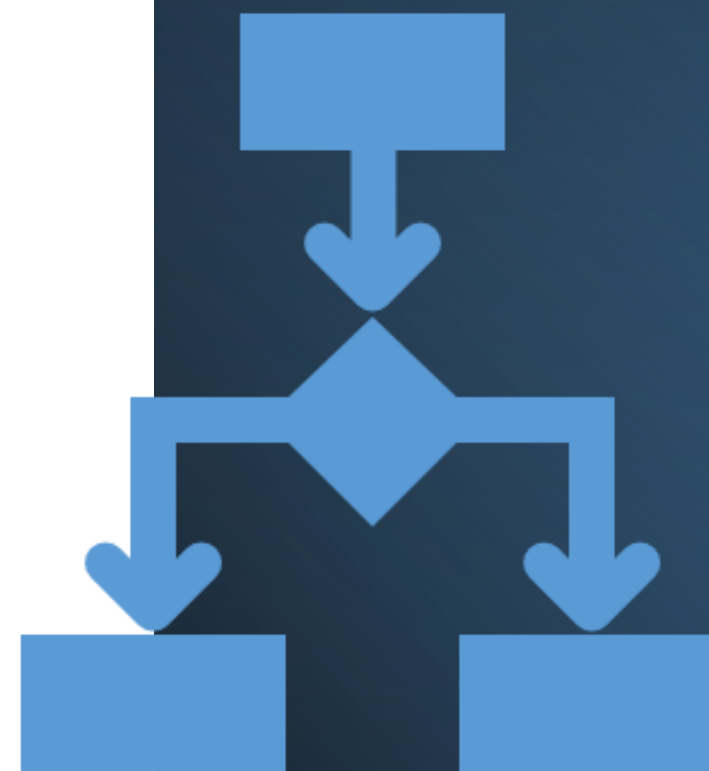
Knihovny

- Obsahují části kódu, které jsou odladěné a lze je použít v našich programech -> cílem je usnadnit a zrychlit programování
- Některé knihovny jsou implementovány v IDE, jiné je třeba importovat
- Pro Arduino jsou knihovny psány objektově v C++ a připojují se do našeho programu přes direktivu **#include**
- Podobně je to i v MicroPythonu, kde se knihovny připojují pomocí příkazu **import**

Řídící struktury

umožňují tzv. větvení (přeskočení) kódu, jeho opakování, apod.:

- if ... else – podmínka
- switch – přepínač
- for – cyklus
- while – cyklus
- do ... while – cyklus

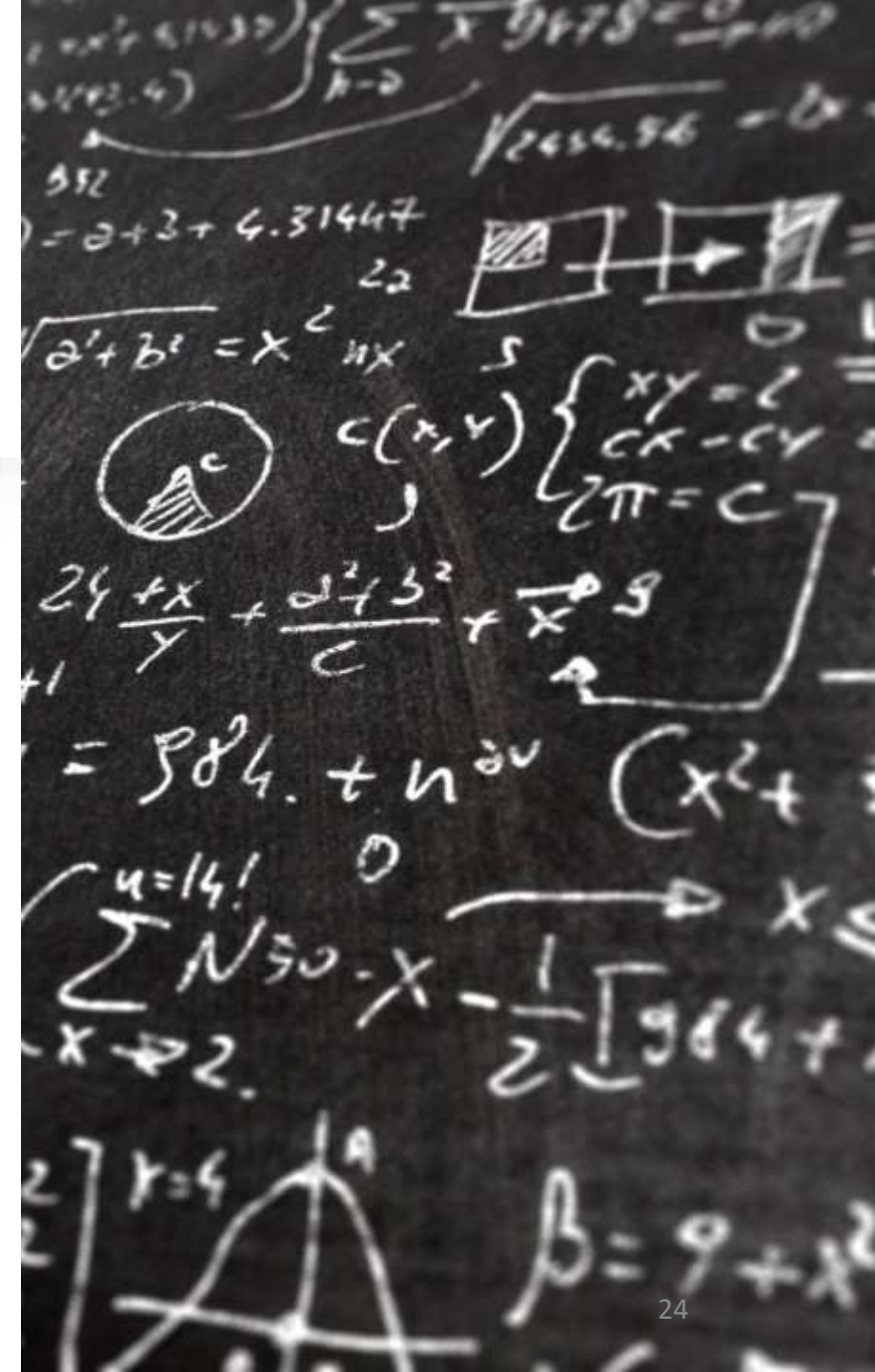


Operátory

- Aritmetické + - * / % =
- Logické ! && ||
- Bitové & << >> ^ | ~
- Porovnání != < <= == > >=
- Zkrácené &= *= ++ += -- -= /= ^= |=

cislo = cislo + 1; cislo += 1; cislo++;

(<https://www.arduino.cc/reference/en/>)



Finální úpravy, psaní dokumentace

- **Komentáře, dokumentace, uživatelský manuál...** - podle potřeby
- Komentáře - mají popisovat, co program vykonává (logiku), problematické části mohou mít důkladnější komentář, např.

```
byte ret = 0; //Definování návratové proměnné
```

```
//Pokud aktuální stav tlačítka (btn) i předešlý stav tlačítka (btnpv) //jsou rozepnuté,  
výsledný stav je rozpnuto (tj. 0)
```

```
if (btn == 1 && btnpv == 1) ret = 0;
```

- Popis knihoven v API (Application Programming Interface)

Základní typy chyb při programování

- **Syntaktická chyba** – chyba v syntaxi/„gramatice“/zápisu – překladač vyhledá a označí
- **Logická chyba** (v logice programu) – program nepracuje přesně jak požadujeme – nutné ladění (zkoušení programu „nanečisto“) – simulátor, emulátor, debugger



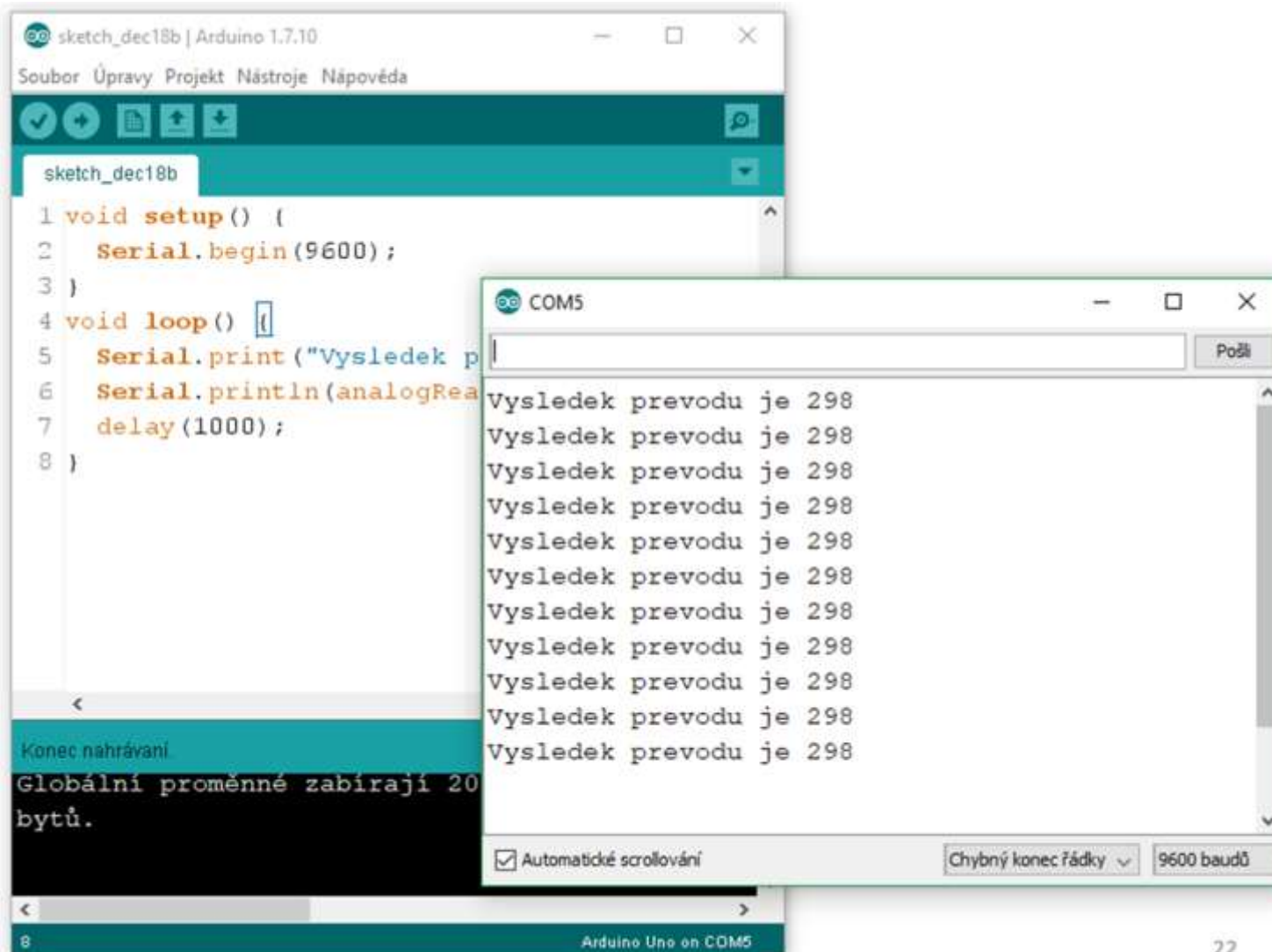
SW vývojové prostředky - ladění programu

- Vývojové prostředí (IDE = integrated development environment) – Arduino IDE:
 - Editor
 - Preprocesor
 - Překladač
 - Linker

avr-gcc a avr-g++

 - Sestavovací program - avr-objcopy
 - Terminál
- Další nástroje (nejsou v Arduino IDE):
 - Simulátor
 - Debugger
 - Virtuální emulátor

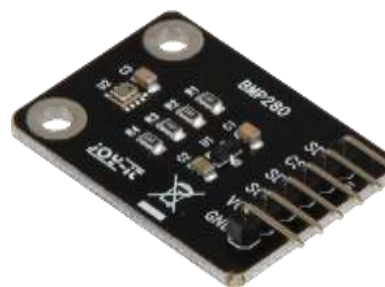
SW vývojové prostředky - psaní a ladění programu



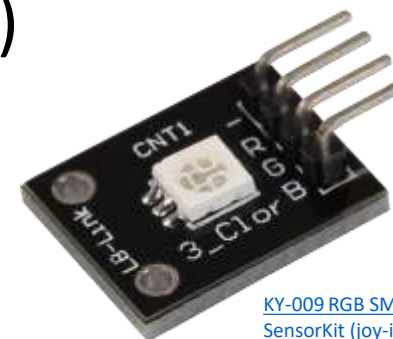
HW vývojové prostředky

- Vývojový kit (Arduino + shiedly a moduly)
- Programátor (většinou součástí Arduina)
- Debugger
- Emulátor
- ...

[KY-052 Pressure sensor / Temperature sensor \(BMP280\) - SensorKit \(joy-it.net\)](#)

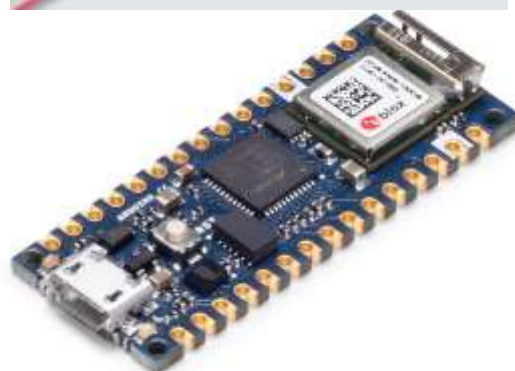


[KY-050 Ultrasonic distance sensor - SensorKit \(joy-it.net\)](#)

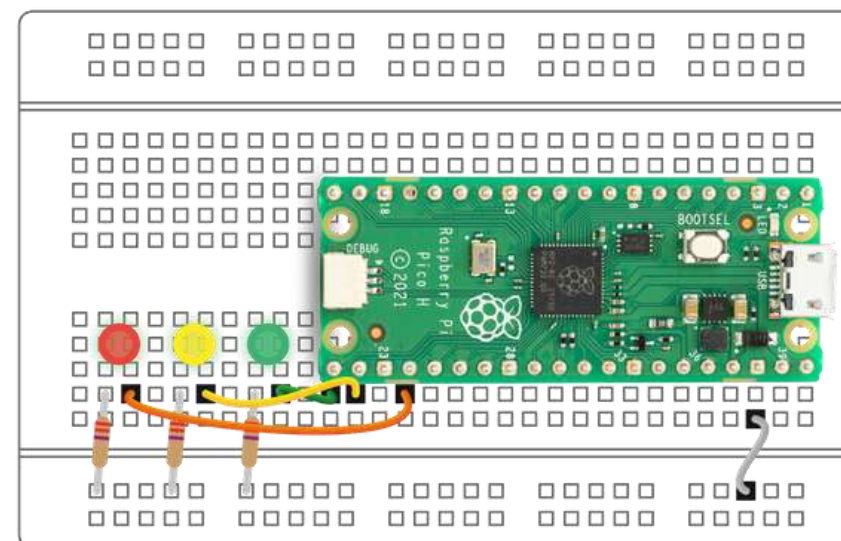


[KY-009 RGB SMD-LED - SensorKit \(joy-it.net\)](#)

[Buy a Raspberry Pi Pico – Raspberry Pi](#)



[Arduino Hardware | Arduino](#)



[Buy a Raspberry Pi Pico – Raspberry Pi](#)

SW zásadní pro programování

- **Kompilátor (překladač)** - překládá program v zařízení, kde byl napsán (např. PC), a kontroluje syntaxi (vyhledává syntaktické chyby)
- **Interpret** – v řídicí jednotce provádí program, tedy vybírá jednotlivé instrukce, které vždy zkontroluje (vyhledá syntaktické chyby), přeloží a "vykoná"
- **Bootloader** – běží ve vyhrazené části paměti programu MCU a používá se pro nahrání uživatelského programu, např. přes sériovou linku, případně jiné sběrnice. Díky tomu lze programovat MCU přes USB kabel, Bluetooth, či Wi-fi
- **Self programming** – schopnost programu „přepsat sám sebe“

